

VeriSafe Agent: Safeguarding Mobile GUI Agent via Logic-based Action Verification

Jungjae Lee^{1,*}, Dongjae Lee^{1,*}, Chihun Choi², Youngmin Im¹, Jaeyoung Wi¹,
Kihong Heo¹, Sangeun Oh², Sunjae Lee^{3,†}, Insik Shin^{1,4,†}

¹KAIST, S. Korea ²Korea University, S. Korea, ³Sungkyunkwan University, S. Korea ⁴Fluiz, S. Korea
{dlwjdw00701,dongjae.lee00,ishin}@kaist.ac.kr

Abstract

Large Foundation Models (LFMs) have unlocked new possibilities in human-computer interaction, particularly with the rise of mobile Graphical User Interface (GUI) Agents capable of interacting with mobile GUIs. These agents allow users to automate complex mobile tasks through simple natural language instructions. However, the inherent probabilistic nature of LFMs, coupled with the ambiguity and context-dependence of mobile tasks, makes LFM-based automation unreliable and prone to errors. To address this critical challenge, we introduce VeriSafe Agent (VSA)¹: a formal verification system that serves as a logically grounded safeguard for Mobile GUI Agents. VSA deterministically ensures that an agent's actions strictly align with user intent before executing the action. At its core, VSA introduces a novel *autoformalization* technique that translates natural language user instructions into a formally verifiable specification. This enables runtime, rule-based verification of agent's actions, detecting erroneous actions even before they take effect. To the best of our knowledge, VSA is the first attempt to bring the rigor of formal verification to GUI agents, bridging the gap between LFM-driven actions and formal software verification. We implement VSA using off-the-shelf LFM services (GPT-4o) and evaluate its performance on 300 user instructions across 18 widely used mobile apps. The results demonstrate that VSA achieves 94.33%–98.33% accuracy in verifying agent actions, outperforming existing LFM-based verification methods by 30.00%–16.33%, and increases the GUI agent's task completion rate by 90%–130%.

*Co-first authors: Jungjae Lee, Dongjae Lee.

†Co-corresponding authors: Sunjae Lee, Insik Shin.

¹The system is available at: <https://github.com/VeriSafeAgent/VeriSafeAgent>



This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License.

ACM MOBICOM '25, Hong Kong, China

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1129-9/2025/11

<https://doi.org/10.1145/3680207.3765248>

CCS Concepts

• Computing methodologies → Mobile agents.

Keywords

GUI Agent, AI Safety; Formal Verification

ACM Reference Format:

Jungjae Lee^{1,*}, Dongjae Lee^{1,*}, Chihun Choi², Youngmin Im¹, Jaeyoung Wi¹, Kihong Heo¹, Sangeun Oh², Sunjae Lee^{3,†}, Insik Shin^{1,4,†}. 2025. VeriSafe Agent: Safeguarding Mobile GUI Agent via Logic-based Action Verification. In *The 31st Annual International Conference on Mobile Computing and Networking (ACM MOBICOM '25)*, November 4–8, 2025, Hong Kong, China. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3680207.3765248>

1 Introduction

The advent of Large Foundation Models (LFMs) [1–3] has revolutionized human-computer interaction, paving the way for a new generation of agents capable of interacting with graphical user interfaces (GUIs) [4–13]. Among these, Mobile GUI Agents stand out for their ability to automate complex tasks within mobile applications, reducing manual effort and enhancing user convenience. By leveraging the reasoning and natural language understanding capabilities of LFMs, these agents can interpret user requests and translate them into sequences of UI interactions [4, 6–8].

However, despite significant advancements, existing Mobile GUI Agents still face fundamental limitations that hinder their reliability and safety in real-world applications. One major challenge stems from the probabilistic nature of LFMs, which can lead to unpredictable and erroneous actions. Additionally, mobile app interactions are often ambiguous and context-dependent, making it difficult even for state-of-the-art LFMs to generate consistently accurate actions [12, 14, 15]. These challenges are especially critical for tasks involving sensitive operations, such as financial transactions or private communications, where errors can have serious or irreversible consequences. Therefore, implementing robust safeguards and verification mechanisms is essential to ensuring the safe and reliable deployment of Mobile GUI Agents.

Recent approaches [8, 16] have attempted to mitigate these challenges by introducing reflection agents that use LFMs

to review the actions of the primary GUI agent and provide feedback. However, these approaches come with significant drawbacks. Since reflection agents also rely heavily on LFM, they remain susceptible to the same errors, leading to a compounding of inaccuracies. Furthermore, repeated LFM queries for reflection incur substantial computational costs and latency, further limiting their practicality. These limitations highlight three key challenges inherent in relying solely on LFM for verification:

1) *Existence of Irreversible and Risky Action*: One way to improve verification accuracy is to analyze both the action and its resulting screen state. While the app screen after the action (*post-action verification*) provides useful context for verification, many critical mobile actions (e.g., making a payment, sending a message) are irreversible. In such cases, verification becomes meaningless, as the agent cannot filter or "undo" the erroneous actions. This fundamentally defeats the core purpose of verification, which prevents critical errors before they occur. To ensure genuine safeguard against critical errors, verification must be performed before executing the action (*pre-action*).

2) *The Forward Assessment Problem*: To Verify an agent's action before its execution, it requires a forward-looking assessment: predicting the effect of the proposed action before execution. However, this process can be as difficult – if not more difficult – than generating the action itself. Furthermore, the predicted outcome aligns with the user's overall goal adds another layer of complexity. Consequently, this approach is akin to solving a simpler problem with a more complex one, potentially turning verification into a performance bottleneck.

3) *Compounding Error Probability*: Mobile tasks typically comprise a sequence of actions, each requiring verification. Continuously relying on LFM for validation at every step increases the likelihood of accumulated errors, which can ultimately degrade overall system accuracy, despite efforts to improve it.

To overcome these challenges, we introduce VeriSafe Agent (VSA), a deterministic, logic-based verification system for Mobile GUI Agents. To the best of our knowledge, this is the first attempt to provide a reliable pre-action verification mechanism grounded in logic-based reasoning rather than existing probabilistic methods, bridging the gap between probabilistic actions and deterministic safety. Specifically, VSA leverages the concept of *autoformalization* [17], which automatically translates natural language user instructions into a formal specification. Using the translated specification, VSA performs runtime verification to ensure the correctness of the agent's actions in runtime.

However, unlike conventional software verification [18–21], which operates on predefined safety requirements, VSA

faces the unique challenge of formalizing impromptu, user-defined instructions on-the-fly, across diverse and dynamic landscape of mobile applications. To accomplish this, VSA introduces three key innovations:

- (1) *Domain-Specific Language (DSL) and Developer Library*¹: A specialized language and accompanying tools tailored to dynamic nature of mobile environments. Collectively, they can encode both the natural-language user instructions and corresponding UI actions as logical formulas.
- (2) *Intent Encoder and Verification Engine*: A system that systematically translates user instructions into logical constraints (expressed as our DSL) and performs runtime, deterministic verification against these constraints.
- (3) *Structured Feedback Generation*: A proactive mechanism that provides actionable feedback to the GUI agent, *before* executing an action, identifying specific logical violations and unmet conditions to guide the agent toward correct task completion.

We implement a prototype of VSA using off-the-shelf online LFM service (GPT-4o) and integrate it with M3A (Multimodal Autonomous Agent for Android) GUI agent [12]. Our evaluation, conducted on 300 user instructions across 18 widely used mobile applications demonstrates that with an estimated ~437 additional lines of code (LoC) per app, VSA successfully verifies GUI agent actions with up to 98.33% accuracy, achieving a false positive rate of 1.3% and a false negative rate of 0.3%. This significantly outperforms baseline reflection agents by 23.2%. Furthermore, VSA's structured feedback, derived from logical verification results, enables the GUI agent to enhance its task completion rate by 130%, a substantial improvement compared to reflection agents, which failed to correct any tasks.

2 Background and Motivation

Graphical User Interface (GUI) Agents. GUI Agents [4–10] are software programs designed to interact autonomously with applications through graphical user interfaces. By interpreting visual and semantic information presented in GUIs and simulating human interactions (e.g., clicks, swipes, and text entry), GUI agents enable automation of digital tasks without requiring internal access to application code or APIs. Recent advancements in artificial intelligence, particularly driven by Large Foundation Models (LFMs), have significantly enhanced the capabilities of GUI agents, enabling them to understand complex user instructions and interpret sophisticated mobile application GUIs. However, due to the inherently probabilistic nature of LFMs and often

¹The library is available at: https://github.com/VeriSafeAgent/VeriSafeAgent_Library

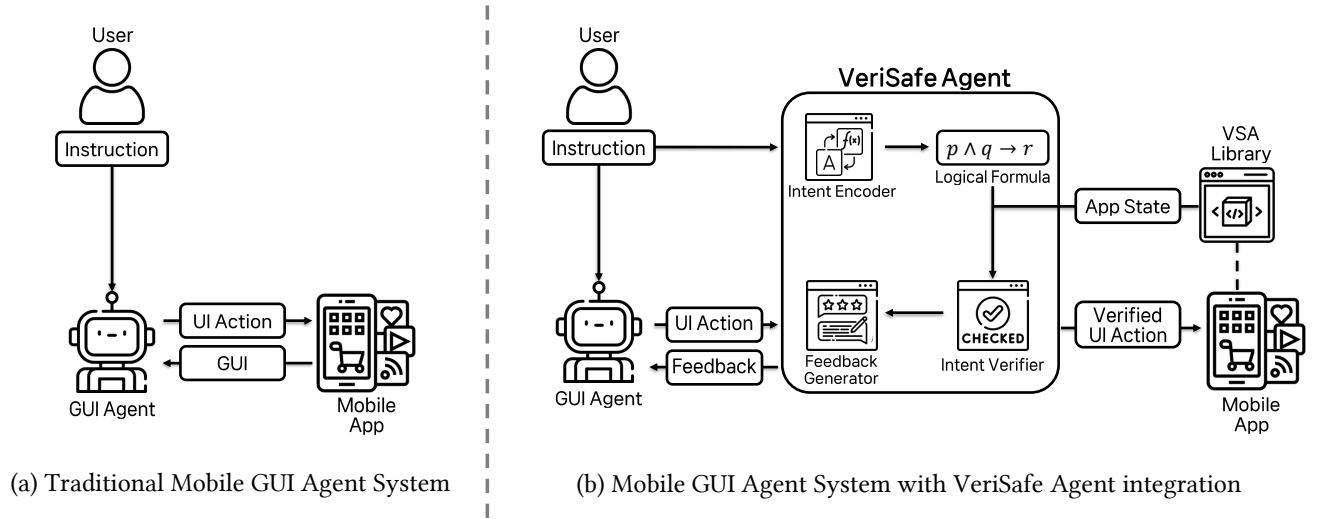


Figure 1: Comparison of (a) a traditional GUI agent system and (b) VSA integrated system.

ambiguous and under-specified nature of GUI interactions, contemporary GUI agents are susceptible to incorrect actions [14, 15, 22–26], necessitating robust verification mechanisms to ensure reliability and safety in practical scenarios.

Safeguarding GUI Agents using Reflection. The current practice for safeguarding GUI agents often involves the use of *reflection agents*, which utilize LFM to review the actions generated by the primary GUI agent. Reflection-based approach can be broadly classified into two categories: pre-action verification and post-action verification.

Pre-action reflection [16] evaluates a proposed action *before* it is executed on the mobile app. This approach serves as an effective guardrail by providing an opportunity to abort or correct erroneous actions before they occur. However, it frequently suffers from low verification accuracy because it requires predicting the outcome of an action. Such inaccuracies lead to high false negatives and positives, paradoxically decreasing the overall system accuracy and undermining the intended safety benefits.

Conversely, *post-action reflection* [8] evaluates an action *after* its execution, leveraging the resulting application state to concretely assess the correctness of the action. While this approach significantly improves verification accuracy, it has a critical limitation: it cannot prevent irreversible actions. For irreversible actions such as financial transactions or sending messages, post-action verification becomes ineffective, as such actions cannot be undone once they are performed.

VSA is designed to address these limitations, providing a reliable *pre-action* verification that is based on a logic-based deduction instead of probabilistic reasoning.

Software Verification for Safety-Critical Systems. Software verification [27–36] encompasses a range of methodologies to ensure that software systems behave correctly according to specification. These formal verification techniques have been widely adopted to provide strong correctness guarantees in safety-critical domains, including hardware design [31], automotive systems [32], and cyber-physical systems [33, 34]. Among them, dynamic verification [35] aims to check the properties of the program or detect incorrect behaviors when the program is executed. The formal definition of malfunctions is written in mathematical expressions such as first-order logic, linear temporal logic, and finite-state automaton. Once the verification properties are defined, verification can be performed by checking whether an automaton reaches an accept state or by converting the problem into a satisfiability modulo theory (SMT) problem to find satisfying variable assignments using SMT solvers [37]. In particular, runtime verification is a type of dynamic verification that monitors the execution of a program in runtime [36]. It detects anomalous behavior through systematic observation of program execution.

3 VeriSafe Agent (VSA): Overview

Inspired by the success of formal verification in other domains, this work adapts techniques from software verification to solve the specific challenges of Mobile GUI Agent safety and reliability.

As illustrated in Figure 1, VSA is layered on top of an existing GUI agent, acting as a verification layer *before* the agent’s proposed UI actions are injected into a mobile app. Given a user instruction, VSA translates it into a logical formula representing the conditions for successful task completion. Then, when the GUI agent generates a UI action, VSA verifies

whether this action satisfies the pre-defined logical formula. If the action is verified as correct, it is passed on to the mobile application. If the action fails verification, VSA provides feedback to the GUI agent, explaining the reason for the failure and guiding the agent to generate a corrected action.

3.1 Challenges

In doing so, VSA addresses three key challenges:

- C1. How to *formally* express the user's intent and the app's execution flow in logically tractable representation?
- C2. How to *accurately* translate natural language user instructions into a formal representation and perform runtime verification against it?
- C3. How to *effectively* communicate verification results back to the GUI agent in an actionable, explainable manner?

C1. Conventional software verification techniques typically rely on precise, pre-defined formal specifications of program behaviors. However, mobile GUI agents operate on natural language instructions, which are inherently informal and ambiguous. Therefore, we need a representation system that can clearly express both the user's intent and the corresponding logical flow of the application to fulfill that intent. To address this, VSA introduces a domain-specific language (DSL, § 4.1) and an associated developer library (§ 4.2). The DSL provides the syntax and semantics for expressing user intent as logical rules that must be satisfied. The developer library enables app developers to explicitly define the app states and transitions required to construct these rules. Together, they allow VSA to represent both the *desired* behavior (user intent) and the *actual* behavior (app execution) in a unified, logically verifiable manner.

C2. Natural language is often vague, underspecified, and prone to misinterpretation, making it difficult to accurately translate into a formal DSL suitable for logical verification. Even the most capable language models often fail to do this without proper external guidance. To address this challenge, VSA employs a *self-corrective encoding* approach (§ 5.1) that enables the language model to iteratively refine its translation based on syntactic and semantic checks. Furthermore, to achieve efficient yet robust verification, VSA provides a two-tiered verification strategy (§ 5.2), adjusting verification intensity according to the action's significance.

C3. Simply flagging an action as incorrect is insufficient for improving the overall practicality of Mobile GUI Agents. To enhance their accuracy and reliability, we must provide actionable feedback that correctly guides agents toward the goal. However, LFM-generated feedback is often vague, underspecified, or even incorrect, potentially leading to repeated errors or suboptimal actions. Therefore, VSA incorporates a structured feedback generation mechanism (Figure 6) that systematically generates precise, rule-based feedback

by identifying unmet or violated conditions based on the logical verification results.

3.2 System Workflow

This section describes the high-level workflow of how VSA logically verifies GUI agent actions.

App Development Phase. To enable rigorous rule-based verification, VSA requires app developers to define the critical states and transitions relevant for verification using VSA developer library (§ 4.2). This includes declaring application state spaces and their update mechanisms. While this requires additional developer effort, typically involving 5 to 10 lines of code per state, and potentially more depending on the application's complexity, such practices are already common in modern app development workflows [38–40].

Intent Encoding. Given a natural language user instruction, the *Intent Encoder* first translates it into structured logical rules capturing essential constraints and logical dependencies within the user instruction. For example, an instruction "Book tickets to the movie M at 7 pm" can be conceptually translated into the following logical rule:

$$\text{MovieInfo}(\text{title}=\text{"M"}, \text{time}=7\text{pm}) \rightarrow \text{Book}$$

where each constraint (e.g., $\text{title}=\text{"M"}, \text{time}=7\text{pm}$) is a condition must be satisfied before booking the ticket.

Verification. When the GUI agent generates a UI action, the VSA developer library (§ 4.2), embedded within the mobile application, intercepts the action and returns the *anticipated* state transition that would result from the execution of this action. The *Intent Verifier* (§ 5.2) then evaluates whether this state transition is valid according to the logical rules generated earlier. For example, if the state transition indicates that the name of the restaurant is 'S' instead of 'R', the `IsNameR` Boolean variable would be evaluated to False, indicating that the action is invalid and we can't proceed to Reserve.

Feedback Generation. Based on the verification result, VSA provides structured feedback to the GUI agent. If verification succeeds, the action is passed on to the mobile app for execution, and *Feedback Generator* (Figure 6) guides subsequent actions. If verification fails, the action is discarded, and *Feedback Generator* explains precisely which predicates or constraints were unmet, guiding the agent toward generating a correct action.

This workflow continues until either the task reaches a final state or the GUI agent explicitly terminates the task despite feedback, resulting in task failure. Through this systematic approach, VSA ensures that Mobile GUI Agents adhere to the logical constraints defined by user instructions while interacting with mobile applications.

<i>Specification</i>	$::=$	<i>Rule</i> *
<i>Rule</i>	$::=$	$Pred \wedge \dots \wedge Pred \rightarrow Objective$ $Pred \wedge \dots \wedge Pred \rightarrow Done$
<i>Pred</i>	$::=$	$Objective \mid StatePred(Constraint^*)$
<i>Constraint</i>	$::=$	$Variable \ Operator \ Constant$
<i>Constant</i>	$::=$	$String \mid Number \mid Boolean$ $Date \mid Time \mid Enumeration$
<i>Operator</i>	$::=$	$= \mid \neq \mid \approx \mid > \mid \geq \mid < \mid \leq \mid \subseteq \mid \not\subseteq$

Figure 2: Simplified syntax of our DSL

4 Domain-Specific Language

This section details the domain-specific language (DSL) and the associated developer library. DSL and developer library are the basis for formally representing user instructions and GUI agent actions. Throughout the remainder of this paper, we illustrate VSA using the running example: “Reserve restaurant *R* before 7 PM. If the restaurant is not available at that time, do nothing.”

4.1 Design of Domain-Specific Language

We first clarify the formal syntax and semantics of our DSL, which enables the formal representation of the user intent and desired app execution flow. Our DSL is designed to flexibly express a wide range of constraints, execution flows, and conditional branches found in natural language user instructions.

4.1.1 Syntax and Semantics. Inspired by logic programming languages (e.g., Datalog [41] and Prolog [42]), our DSL is structured around Horn clauses. A Horn clause refers to a logical formula with the specific structure $p_1 \wedge p_2 \wedge \dots \wedge p_n \rightarrow o$, where $p_1, p_2, \dots, p_n, o$ are predicates evaluating to boolean values. Horn clauses are optimized for representing objective attainment conditions and execution flow, commonly used in program verification [43–46]. Each Horn clause signifies that the preconditions for o to be satisfied are $p_1, p_2, \dots, p_n$. The formal syntax of our DSL is summarized in Figure 2.

Specification and Rule. In our DSL, a single user instruction is expressed as a set of Horn clauses called *specification*. Each Horn clause is called a *rule*, which represents a list of predicates (preconditions) and the objective to be achieved (conclusion). Rules break down a user instruction into multiple smaller steps, making it easier to verify complex instructions incrementally. Each rule consists of predicates and an objective which represent the precondition and intermediate step, respectively. Predicates are composed of state predicates and objectives of the other rules. State predicates specify the conditions that must hold in the application for the intermediate step to be achieved. When the objective of another rule is used as a predicate, it establishes a precedence relationship between rules. A rule is satisfied if all predicates

```

R1 :RestaurantInfo(name = “R”)
      ∧ ReserveInfo(date = Today, time < 19:00, available = True)
      → Reserve

R2 :Reserve ∧ ReserveResult(success = True) → Done

R3 :RestaurantInfo(name = “R”)
      ∧ ReserveInfo(date = Today, time < 19:00, available ≠ True)
      → Done

```

Figure 3: Specification for the instruction “Reserve restaurant *R* before 7 PM. If the restaurant is not available at that time, do nothing.”

hold. If a rule ends with Done and such a rule is satisfied, the task is considered complete.

State Predicate. State predicates represent abstract states of applications. They ignore overly specific information (e.g., size of buttons) and represent only the information important for verification. A state predicate p takes the form $p(c_1, \dots, c_n)$, where each c_i is a constraint. Each constraint allows us to express a target state precisely. For example, a state predicate of RestaurantInfoResult can have constraints on restaurant_name, location, and cuisine_type.

Constraint. A constraint c has the form $var \ op \ const$ (e.g., $x = 10$), where x is a variable representing a specific element within the application state (e.g., restaurant_name, location). op is a comparison operator, and $const$ represents a constant value. The expression $x; op; const$ returns true if the variable x meets the condition defined by the operator op and the constant $const$. If x has not been observed in the app, it defaults to *undefined*, making the constraint always false. The variable x can be one of the following six types: string, number, Boolean, enumeration, date, and time. For example, given a constraint $x \geq 10$, if the value of the variable x was observed to be 15 in the application, then the constraint is considered true.

4.1.2 Running Example. Consider our running example of reserving a restaurant. The instruction is translated into the specification shown in Figure 3. RestaurantInfo is a predicate that represents conditions on information about the restaurant you want to reserve, with the constraint that the name must be “R”. ReserveInfo is information about the reservation, with the selected date, time, and availability as constraints. The first rule, R_1 , means to make a reservation if the restaurant name is “R” and the reservation is available before 19:00 today. The second rule, R_2 , indicates task completion if the reservation is confirmed to be successful. The last rule, R_3 , represents another task completion if the reservation is not available before 19:00 today.

Notice that the objective Reserve of rule R_1 appears as a predicate in rule R_2 . This signifies that R_1 must be satisfied before proceeding to R_2 .

Specification is dynamically evaluated during the agent's interaction with the application. Initially, all variables x are considered *undefined*. The values of these variables are updated at runtime by the state transitions caused by the GUI agent's actions. These updates are governed by pre-defined state update methods written in the developer library (see § 4.2). Whenever a variable's value changes, the associated constraint is re-evaluated.

For example, consider the `ReserveInfo` predicate. If the GUI agent attempts to reserve restaurant "R" at 18:00 today, the update rule will set the values of the date and time variables to Today and 18:00, respectively. When we evaluate the constraints against the updated variables, `date=Today` and `time<19:00` will both be true. If availability is observed to be true in the app state after selection, all constraints of the `ReserveInfo` predicate are true, and the predicate itself becomes true. This runtime evaluation of the specification allows VSA to continuously monitor the agent's progress and verify whether its actions are leading the application toward a state that satisfies the user's intent.

4.2 Developer Library

To effectively generate *specification* from a user instruction and evaluate it against agent's actions, VSA requires a definition of two key components: (i) the set of candidate state predicates (i.e., the application's state space) that serve as building blocks for the specification, and (ii) the mechanisms through which each predicate's constraint variables x are updated during app execution (i.e., state transitions).

To this end, VSA provides a *developer library* that empowers app developers to explicitly define application states (as state predicates) and their corresponding update mechanisms (as state transitions). This approach capitalizes on developers' deep understanding of their application's internal logic and intended behavior. Although it requires additional developer effort, it enables highly accurate state definitions suitable for rule-based logical verification—a level of precision that is unattainable with automated approaches like static code analysis or LFM-driven dynamic analysis.

Furthermore, this approach aligns with established practices in modern mobile app development. Frameworks like React Native (state management [38]), Android's Jetpack Compose (state hoisting [39]), and iOS's SwiftUI (@State [40]) all encourage developers to manage app state and transitions. By closely mirroring these familiar paradigms, VSA ensures seamless, intuitive, and minimally invasive integration into standard app development workflows.

4.2.1 States Definition. A *state*, in the context of the VSA Library, is characterized by a set of typed variables (as defined in § 4.1.1) that capture essential details relevant to the application's current context—each variable corresponds to

the constraint of the state predicate. Each state definition includes i) *name*, ii) *description*, and iii) *variables*:

Developers can define these states using an external JSON file or provided APIs (i.e., `defineState()`):

```
{
  "name": "RestaurantInfo",
  "description": "Information about the
    restaurant you want to reserve.",
  "variables": [{"name": "String"}]
}
```

These defined states are then used as candidate state predicates when VSA's Intent Encoder (§ 5.1) translates user instruction into a specification.

4.2.2 Pre-action State Update Triggers. VSA enables developers to define *triggers* specifying when and how state variables are updated. Triggers correspond directly to meaningful state transitions within the mobile application. Most triggers are associated with UI interaction handlers (e.g., `onClickListener`) or critical points in the application logic.

Triggers return the *expected* state update before executing the action. The returned update is used to verify the action. This ahead-of-time update provides two key benefits: i) It prevents irreversible actions (e.g., financial transaction, sending a message) from being executed if they violate the user's intent; ii) It allows the GUI agent to easily retry with a different action without having to revert the effects of a previous, incorrect action (e.g., deleting incorrect text input).

To facilitate pre-action updates, the VSA developer library provides wrapper listeners for common UI input handlers. These wrappers intercept user interactions and update the relevant state variables according to the developer-defined logic *before* executing the original event handler's code. The original 'onClick' code which performs the actual operation is executed only if the verification succeeds. For example, consider the 'RestaurantInfo' state predicate defined earlier. A developer could associate a trigger with the search button's click listener as follows:

```
searchButton.VSAOnClickListener() -> {
  vsaStateManager.updateState("RestaurantInfo", {
    "name": searchText.getText(),
  });
  /* existing code for onClickListener */
};
```

Another key advantage of developer-defined state predicates is the control over *granularity*. In many mobile tasks, individual atomic actions are not inherently meaningful. For example, in an e-commerce application, entering individual fields of a shipping address (street, city, zip code) is a set of preparatory steps. The critical state change is the *submission* of the complete address. Verifying every single atomic action would be inefficient and could mislead LFMs to think that

some steps are incomplete. By allowing developers to strategically define verification checkpoints that have a meaningful impact on the task's execution, VSA can significantly improve the efficiency and accuracy of task verification.

4.2.3 Post-action State Update Triggers. While pre-action verification is ideal for preventing errors, some action outcomes are inherently unpredictable (e.g., the result of a network request). For these scenarios, VSA supports *post-action* state updates. Developers can call the 'updateState()' API directly, often within asynchronous callbacks, to update the state after the outcome of an action is known. This allows developers to handle dynamic application behaviors and real-world uncertainties such as network delays, asynchronous operations, error conditions, and dynamically determined content.

For instance, consider a flight booking app where the price and availability of flights change dynamically based on real-time data. A developer can define a `FlightSearchResult` state with variables for price, availability, and departure time. They can then associate a state update trigger with the asynchronous network callback that gets executed when the user clicks the "Search" button. Upon receiving the network response, the developer-defined trigger logic updates the `FlightSearchResult` based on the actual results. If the network fails or no seats are available, the trigger can handle these conditions appropriately, updating the state to reflect the error or unavailability. This ability to embed complex, conditional logic directly into the verification framework enables VSA to handle a wide range of user requirements and edge cases.

5 Verification Engine

5.1 Intent Encoder

Natural language instructions cannot be directly used for formal verification; a formalization process is required. We optimize the LFM-based autoformalization method in the context of mobile agents to automatically convert instructions into specifications written in our DSL. Relying solely on LFM's output poses risks due to their probabilistic nature and potential for hallucinations. To address this limitation, we applied *self-corrective encoding*, which corrects errors automatically, and *experience-driven encoding* techniques that gradually adapt to each app.

5.1.1 Self-Corrective Encoding. To ensure the encoded specification is correct and includes all relevant constraints, VSA employs two complementary self-corrective techniques: (i) rule-based syntax checking, and (ii) decoding-based semantics checking. These methods allow the LFM to incrementally identify, correct, and refine its translations, improving accuracy and reliability.

As an initial step, the *Intent Encoder* prompts the LFM with: i) the user's natural language instruction and ii) the set of developer-defined states (from the developer library). From this information and user instruction, the Intent Encoder generates a draft specification representing the user's intent using available states. The draft specification then undergoes two checking stages:

Rule-based Syntax Checking. Inspired by conventional programming languages, VSA checks that the generated Specification strictly adheres to the DSL's syntax (§ 4.1.1) and that the types of constants c used within the Specification are consistent with the types of variables declared in the developer-defined states. For example, if the LFM generates a state predicate like `RestaurantInfo(name ≥ 100)` while the type of the name variable is defined as 'String' in the developer library, this inconsistency is immediately flagged as a syntax error. If an error is detected, a structured error message is returned to the encoding LFM as feedback, guiding it toward a corrected translation.

Decoding-Based Semantics Checking. To further validate the semantic correctness of the translation, the generated Specification is *decoded* back into a natural language description using a separate *decoder LFM*. A *checker LFM* then compares this decoded description to the original user instruction, confirming that the encoding genuinely captures the user's intent rather than merely manipulating symbols. If discrepancies are found, the checker LFM identifies the root cause and provides feedback to the *encoding LFM*.

These complementary checking mechanisms significantly improve the robustness and accuracy of the intent encoding process, effectively mitigating risks associated with solely relying on probabilistic LFM outputs.

5.1.2 Experience-driven Encoding. Although *Self-corrective Encoding* significantly enhances translation accuracy, it occasionally produces hallucinations when presented with an enormous number of candidate state predicates.

To address this challenge, VSA introduces an *Experience-driven Encoding*, which leverages previously successful encodings to guide subsequent translations. When an instruction is successfully translated and verified, its result specification is cached. Since a single specification consists of multiple rules representing an instruction's intermediate objectives, numerous instructions for the same application may share similar rules. By retrieving and prioritizing predicates based on past encoding experience, we can significantly reduce the search space for the LFM encoder, improving both efficiency and consistency.

5.2 Intent Verifier

The verification process is straightforward. When the GUI agent generates an action, the associated state updates (defined via the developer library, § 4.2) are triggered. They

update the values of the variables x of the corresponding constraints and state predicates in the Specification. After each update, the affected predicates are re-evaluated to a new truth value.

Based on this basic mechanism, VSA performs two levels of verification: *Predicate-level Verification* at the predicate level and *Rule-level Verification* at the rule level.

Predicate-level Verification is performed at *every* state update. Whenever a state predicate within the Specification is re-evaluated due to a variable update, its truth value is checked. If a predicate evaluates to false, this indicates a *potential* violation of the user's intent. For example, if the Specification includes the predicate `RestaurantInfo(name = "R")`, and the agent's action causes the name variable to be updated to "S", the predicate would evaluate to false, indicating a *potential* violation of user intent.

It is important to note that an incorrect predicate at this stage does not necessarily mean a definitive error because the updated state may represent an intermediate step necessary to eventually reach the final, correct state. For example, when buying three apples, the number of apples might be incremented one at a time. Each intermediate update (1 apple, then 2 apples) would result in a false value for a predicate requiring three apples, until the final update.

Therefore, upon detecting a failed verification, VSA reverts the predicate update and provides the GUI agent with *soft feedback* (Figure 6). This feedback serves as a warning, indicating a potential deviation from the intended path, and encourages the agent to double-check its subsequent actions.

Rule-level Verification is performed at irreversible or critical checkpoints within the task execution. To identify these checkpoints, we modified the GUI agent's prompt so that it generates a *critical* flag along with the action if that action is intended to directly achieve one of the objectives o within the Specification (e.g., Reserve).

When the GUI agent generates a *critical* action, VSA checks for the complete satisfaction of the conditions required to achieve a specific Objective o . If any predicate in the corresponding rule is unsatisfied, the action is considered invalid and blocked. Hard feedback is provided to the agent, detailing the specific unmet conditions. This prevents the execution of potentially irreversible or incorrect actions that would definitively violate the user's intent.

This two-tiered verification approach guides the agent toward its objectives while ensuring the safe execution of high-risk actions.

6 Structured Feedback Generator

The effectiveness of a verification system hinges not only on error detection but also on its ability to guide the agent towards correct behavior. This section explains how VSA generates structured feedback based on verification results.

F_1 : "To perform Reserve, 1. RestaurantInfo that represents Information about the restaurant you want to reserve should have 'name' equal to "R"; 2. ReserveInfo that represents reservation details should have 'date' equal to "Today" and 'time' less than 19:00, and 'available' equal to True. So far, you have achieved step 1."

F_2 : "To complete the task, 1. perform Reserve 2. ReserveResult that represents reservation result should have 'success' equal to "True"."

F_3 : "To complete the task, 1. RestaurantInfo that represents Information about the restaurant you want to reserve should have 'name' equal to "R"; 2. ReserveInfo that represents reservation details should have 'date' equal to "Today" and 'time' less than 19:00, but 'available' not equal to True. So far, you have achieved step 1."

Figure 4: An example of roadmap feedback. Each F_i corresponds to Rule R_i in the Specification

VSA provides three distinct types of feedback: i) Roadmap Feedback, ii) Predicate-level Soft Feedback, and iii) Rule-level Hard Feedback.

Roadmap Feedback. VSA provides roadmap feedback that outlines the overall path toward task completion regardless of verification outcome. Because the logical Specification of VSA explicitly encodes the preconditions required for achieving the user's objective, it naturally serves as a comprehensive guideline for the GUI agent.

Specifically, the Feedback Generator converts each rule in the Specification into a natural language explanation, detailing which predicates must be satisfied for each objective. These explanations incorporate developer-defined descriptions for each state predicate, constraints, and currently satisfied predicates. For example, Figure 4 demonstrates the roadmap feedback for the restaurant reservation scenario after successfully searching for restaurant "R".

Unlike approaches relying on ambiguous action histories or abstract planning, this structured guidance clearly communicates both current progress and next objectives.

Predicate-level Soft Feedback. When predicate-level verification fails (i.e., a state predicate evaluates to false), VSA provides feedback indicating that the agent's action *may* be incorrect. This feedback includes a description of the desired state for the violated predicate, allowing the agent to reconsider its action. Because predicate-level errors could represent intermediate steps rather than genuine errors (as described in § 5.2), soft feedback is advisory rather than strictly prohibitive. If the GUI agent generates the same action even after receiving the feedback, VSA permits the action, acknowledging its non-critical nature.

Rule-level Hard Feedback. For critical actions subject to rule-level verification, VSA provides comprehensive feedback at the rule level, detailing all state predicates that remain unfulfilled. Unlike soft feedback, if the GUI agent generates the same action again after receiving hard feedback, VSA will

not allow the action to proceed. Instead, it enforces that all required state predicates must be satisfied before permitting the critical action to be executed.

Through these structured, deterministic feedback mechanism, VSA effectively guide GUI agents toward correct task completion, significantly outperforming existing reflection-based methods. By eliminating ambiguity and providing clear, actionable guidance grounded in formal logic, VSA enhances the reliability, safety, and practicality of automated mobile task execution.

7 Implementation

Our VSA implementation is designed as a modular component that can be seamlessly integrated with existing GUI agents without requiring modifications to their underlying architecture. VSA is implemented using Python, and its Intent Encoder leverages off-the-shelf LFM GPT-4o.

Developer Library Guidelines. Developers should carefully determine the granularity of State Predicates to suit their application context. Overly fine-grained State Predicates can make the encoded specification too detailed, increasing the false positive rate. Conversely, coarse-grained State Predicates may introduce loopholes in the encoded specification, leading to a higher false negative rate. To assist with finding the sweet spot, the VSA Library provides detailed documentation and guidelines on how to effectively define State Predicates. Given the potential severity of false negatives, it's recommended to begin with a fine-grained verification first and then iteratively refine the predicates to minimize false positives.

String Similarity. To compare strings semantically rather than structurally, we use OpenAI's text-embedding-3-small model [1] and compute cosine similarity. Based on empirical studies, we set a similarity threshold of 0.7 to determine equivalence.

8 Evaluation

In this section, we evaluate the performance of VSA against baseline guardrail frameworks in terms of *i) verification accuracy*, *ii) feedback effect*, and *iii) latency and cost*. Throughout the experiments, we integrated each framework, including VSA with the M3A mobile GUI agent [12]—a simple yet powerful mobile GUI agent powered by GPT-4o—and conducted evaluations using a Google Pixel 8 smartphone.

Baselines & VSA Variants. To evaluate VSA, we compare it with two LFM-based reflection schemes: Pre-action reflection [16] and Post-action reflection [8], both of which utilize GPT-4o as their backbone LFM. For VSA, we evaluate two variants: VSA-cold and VSA-warm. VSA-cold encodes user instructions solely using self-corrective encoding, without cached memory. On the other hand, VSA-warm leverages

both self-corrective encoding and experience-driven encoding, assuming that a cached list of candidate predicates is available for the given instruction (see § 5.1 for details).

8.1 Dataset

To comprehensively evaluate VSA, we constructed a dataset¹ of 300 user instructions, divided into two complementary subsets of 150 instructions each: the *Correct* dataset and the *Wrong* dataset.

Each instruction is paired with an execution path—a sequence of UI actions performed by the GUI agent. In the *Correct* dataset, instructions are matched with execution paths that accurately fulfill the user's intent. In contrast, the *Wrong* dataset deliberately introduces mismatches between instructions and their corresponding execution paths. Specifically, we altered key details in the original *Correct* instructions to create discrepancies. For example, "*On Play Books, turn to the search page and check for top-selling books*" was modified to "*On Play Books, turn to the search page and check for newly released books*", making the original execution path no longer satisfy the updated instruction. Such cases test the verification framework's ability to detect errors, enabling evaluation of both True Positives (correctly rejecting invalid executions) and False Negatives (incorrectly accepting invalid executions).

Of the 150 unique instructions, 125 were sampled from the LlamaTouch dataset [15], selected based on app popularity (global usage), instruction realism (likelihood of real-world occurrence), and current app availability. We excluded tasks that were overly localized, deprecated, or overly specific and prone to overfitting. The remaining 25 instructions were newly created to form the *Challenge* dataset, designed to push the system to its limits with longer action sequences, multiple constraints, and conditional branches. Spanning 9 different applications, these instructions average 19.16 steps (maximum 40), far exceeding LlamaTouch's average of 5.67 steps.

Finally, as described in § 4.2, VSA relies on developer-defined state specifications and state-update triggers. To emulate this in closed-source mobile apps, we manually annotated state predicates and state transitions for each GUI screen and action in the dataset. This manual annotation serves as a surrogate for direct integration with the developer library, effectively replicating the library's intended functionality.

8.2 Verification Accuracy

Figure 5 presents the verification accuracy of different verification methods, evaluated using standard metrics: Accuracy, Precision, Recall, and F1 Score. Notably, VSA consistently

¹The dataset is available at: <https://github.com/VeriSafeAgent/VeriSafeAgent>

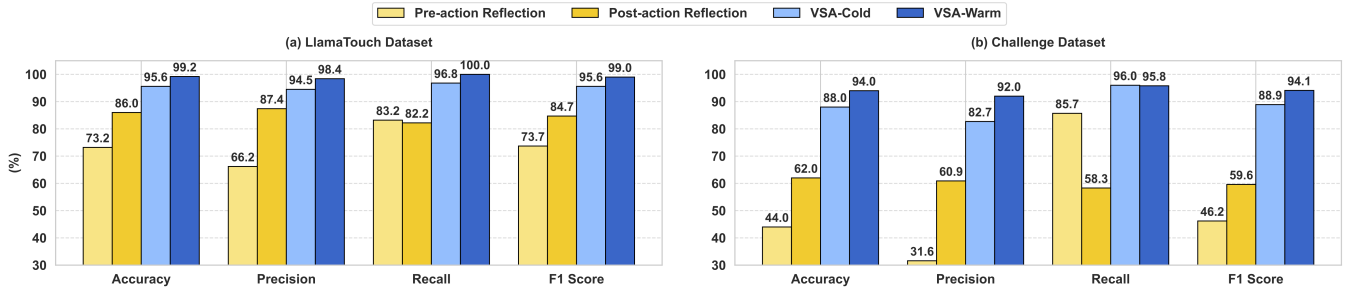


Figure 5: Verification accuracy

Error Type	Variant	LlamaTouch	Challenge
Overall Missing Predicate	VSA-Cold	76/250	24/50
	VSA-Warm	24/250	14/50
Critical Missing Predicate	VSA-Cold	10/250	3/50
	VSA-Warm	3/250	2/50
Superfluous Predicate	VSA-Cold	7/250	4/50
	VSA-Warm	0/250	0/50
Constraint Mismatch	VSA-Cold	8/250	11/50
	VSA-Warm	4/250	6/50

Table 1: VSA predicate translation accuracy

outperforms reflection-based methods across all metrics and datasets, achieving near-perfect accuracy on the LlamaTouch dataset.

For simple tasks in LlamaTouch dataset, post-action reflection achieves significantly higher accuracy compared to pre-action reflection. This is because it leverages the action’s resulting screen as additional context for verification. However, it has critical limitations: it cannot correct irreversible actions. Moreover, our results indicate that this advantage diminishes significantly as instructions become more complex and involve longer action sequences (as seen in the Challenge dataset). This trend highlights the compounding effect of errors stemming from over-reliance on LLMs for verification.

In contrast, VSA performs *pre-action* verification while maintaining high performance across both datasets, thanks to its design that encodes the specification once and performs subsequent verification in a rule-based manner, regardless of task complexity or length. Notably, VSA-Warm achieves near-perfect results on LlamaTouch and significantly outperforms all baselines on Challenge, highlighting the benefit of a well-constructed predicate memory. While building such a memory (e.g., via offline exploration, user demonstrations, runtime analysis) is beyond this paper’s scope (§ 10), these results underscore its promise.

An interesting observation is the high recall of Pre-action Reflection on the Challenge Dataset. However, this comes at the cost of a high false positive rate (72%), leading to excessive error flagging when instructions become complex. While this increases the chance of detecting *any* error, the

likelihood that a flagged action is truly erroneous remains low (low precision).

Predicate Translation Accuracy. To gain deeper insight into the verification results, we further evaluated VSA at the state predicate level. We manually constructed a ground-truth set of predicates P_{GT} for each instruction and compared it against P_{Spec} generated by VSA.

VSA’s Verification errors fall into three categories: 1) **Missing Predicate** — necessary predicates absent from the specification, potentially allowing incorrect behaviors (False Negatives); 2) **Superfluous Predicate** — unnecessary predicates included, potentially rejecting correct behaviors (False Positives); 3) **Constraint Mismatch** — correct predicate but with inaccurate constraints, which can cause both False Negatives and False Positives.

Table 1 shows the counts of such errors. Note that *Overall Missing Predicate* counts all predicates absent relative to P_{GT} , which defines the maximal set of possible predicates (including semantically overlapping ones). In practice, VSA tends to select a minimal set of core constraints, which can lead to an overestimation of “missing” predicates. We therefore also report *Critical Missing Predicates*, which are those whose absence directly affects verification results.

A key observation is that VSA-Warm significantly reduces both missing and superfluous predicates. This demonstrates that predicate memory effectively improves translation accuracy by narrowing the search space for necessary predicates. Furthermore, the distribution of errors across all three types indicates that, despite VSA’s strong overall verification performance, residual risks from predicate translation remain. In the Discussion section, we propose complementary policies to mitigate these risks and further enhance VSA’s safety.

Randomness Mitigation. Although VSA issues only a single LFM query per task, repeated verification of the same instruction may yield inconsistent results due to the stochasticity of LLMs. The same inconsistency applies to all methods that invoke an LFM, including reflection-based approaches. To mitigate this randomness, we employ a majority-voting scheme that repeats verification N times and returns the majority decision (*Majority-at-N*).

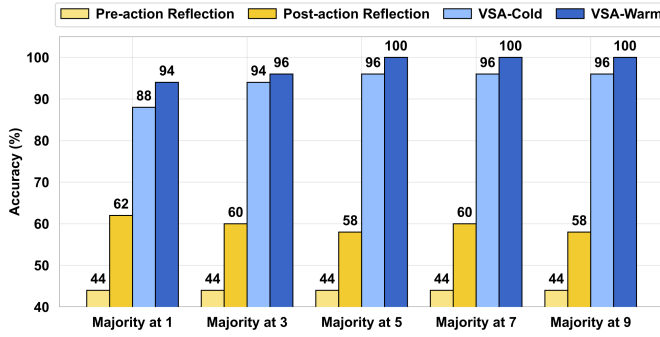


Figure 6: Verification accuracy with *Majority-at-N*

Figure 6 reports accuracy on 50 instructions from the Challenge dataset as N increases. Reflection-based schemes show limited gains and remain less accurate overall, whereas both VSA-Cold and VSA-Warm improve substantially, reaching 96% and 100%, respectively.

This performance disparity stems from differences in verification mechanisms: reflection schemes produce direct true/false judgments with relatively low variance, whereas VSA encodes a logical specification with multiple predicates—small stochastic differences can flip individual predicate decisions—making majority voting especially effective at averaging out noise.

These findings indicate that VSA can achieve substantial performance gains through majority voting. However, increasing verification iterations introduces considerable overhead in terms of latency and cost. We analyze this trade-off in detail in § 8.4.

8.3 Effectiveness of Feedback Generation

The usefulness of a verification system lies not just in identifying errors, but in improving the GUI agent’s performance through feedback generation. This section evaluates VSA’s logic-based feedback generation against the LLM-based feedback used in reflection methods.

We instructed the M3A GUI agent to execute 25 challenging instructions from our Challenge dataset. Initially, without external feedback, M3A successfully completed only 10 of 25 tasks (40% success rate). We then integrated each verification system with M3A and re-executed the failed instructions to observe whether the feedback can correct the previously failed tasks.

Figure 7 summarizes the results. Notably, Neither pre-action nor post-action reflection corrected any previously failed tasks. In contrast, VSA-Cold corrected 9 of 15 failures (60%), and VSA-Warm corrected 13 of 15 (86%). The two remaining failures under VSA-Warm were due to unconventional app designs rather than misunderstanding of context. For instance, one clock application required time input as a single numerical sequence (e.g., "143000" for 14:30:00) rather than separate hour, minute, and second components. Such

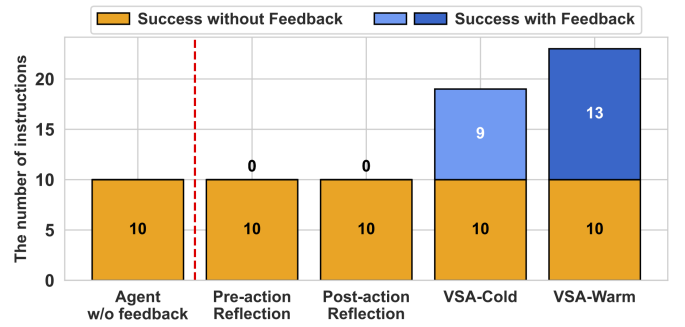


Figure 7: Effect of feedback on task completion

unconventional interfaces could limit VSA’s ability to provide meaningful feedback. Nevertheless, given that M3A employs GPT-4o—one of the most capable LLMs—achieving this performance improvement through systematic harness demonstrates VSA’s effectiveness.

VSA’s superior performance stems from its ability to generate precise, deterministic feedback that clearly identifies what needs correction (i.e., unmet predicates). Moreover, VSA’s roadmap feedback proactively guides agents toward correct execution paths, preventing errors from happening in the first place. For example, when asked to upvote the most controversial comment in a subreddit, M3A without feedback repeatedly toggled the upvote button because it could not identify whether the upvote has been properly set. However, when VSA provided clear guidance that the precondition to set ‘upvote’ to ‘true’ had been satisfied, M3A completed the task without unnecessary repetition.

By contrast, reflection methods rely on LFM-generated narrative feedback that is often misleading or inconsistent; we observed cases where a reflection method initially flagged an action as incorrect but reversed its judgment when the agent repeated the same action, indicating indecisive and unreliable guidance.

A potential limitation of VSA, though not observed in our evaluation, is that false positives could prematurely terminate tasks that the agent might otherwise complete successfully. Unlike reflection-based methods, where agents typically recover within 1–3 actions after misleading feedback, a single incorrect predicate in VSA’s specification could halt task execution entirely. However, our evaluation demonstrates that VSA’s decisive feedback provides practical benefits that significantly outweigh the risks of rare false positives. In safety-critical applications such as automated mobile tasks, preventing erroneous actions—even at the cost of occasional false positives—is preferable to allowing potentially harmful actions to proceed.

8.4 Latency and Cost

VSA not only enhances verification reliability but also significantly reduces latency and cost overhead. Figure 8 plots

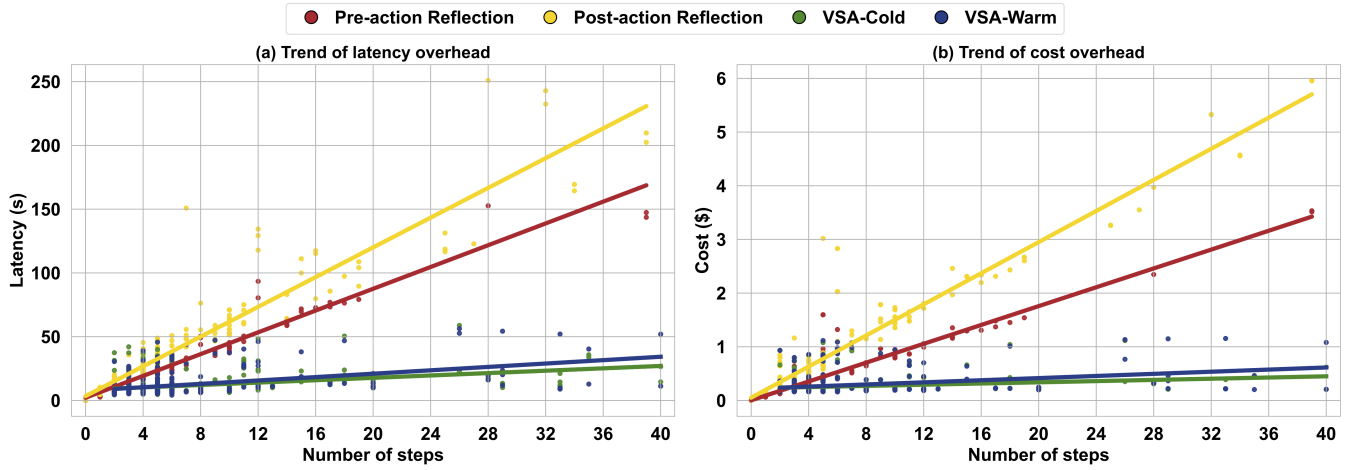


Figure 8: Latency and cost overhead of verification methods with respect to the number of action steps.

the additional latency and cost incurred by verification as a function of task length (number of steps). As shown by the fitted lines, VSA consistently maintains lower overhead than reflection-based methods across all task lengths. Notably, because reflection-based approach requires an additional LFM query for each action, its overhead scales proportionally with task length. While not depicted in the graph for clarity, its overhead is comparable to that of the GUI agent itself, effectively doubling the total expense.

In contrast, VSA shows near constant overhead, independent of task length. This is because VSA uses LFM only once – when encoding user instructions into formal specifications. All subsequent action verifications are performed using rule-based logic. Although Figure 8 shows a slight upward trend in VSA’s overhead for longer tasks, this increase is attributed to more complex instructions naturally producing longer specifications with additional constraints. However, this increase remains trivial compared to the $O(N)$ complexity of reflection-based methods.

GUI Agents inherently incur substantial latency and cost due to their reliance on complex cognitive reasoning. As shown in Figure 8, tasks exceeding 10 steps can cost over \$1 and take nearly a minute to complete. Therefore, minimizing this overhead is as critical for practical deployment as achieving high accuracy. VSA thus proves to be more suitable and practical for real-world deployment across all dimensions compared to existing approaches.

8.5 Developer Burden

To estimate the developer effort required for VSA integration, we analyzed 18 commercial mobile applications from our dataset. Since these applications are closed-source, we simulated the integration process by manually enumerating the verification-relevant state predicates for each of the 18 target

apps. We then estimated the lines of code (LoC) by multiplying predicate counts by the average LoC per predicate measured from our reference implementation.

On average, each application required 58.77 predicates, translating to approximately ~437 LoC. Across all 18 apps, we generated 1,058 predicates in total, of which 540 (51%) were activated during our experiments. The number of predicates varied with application complexity. Google Tasks, a single-function app, required only 22 predicates, while feature-rich applications like Instagram, X (Twitter), and Facebook required 81, 91, and 72 predicates, respectively. This correlation between application complexity and predicate count is expected, as more complex applications naturally have more states to verify.

While these estimates suggest a non-negligible implementation effort, several factors mitigate the burden. First, many predicates follow standard patterns that could be templated or automated. Second, the predicate definitions align naturally with existing application state logic that developers already maintain. Third, predicate granularity provides a practical lever for balancing coverage and effort, allowing developers to selectively implement only safety-critical predicates.

Overall, the effort required for predicate definition represents a one-time investment that yields ongoing benefits in error prevention and agent reliability. For applications where automated agents perform high-stakes tasks, the safety improvements demonstrated in our experiments justify this initial development effort.

9 Related Work

Validating LFM responses. The probabilistic nature of LFM often leads to hallucinated or inaccurate content. Numerous methods have been proposed to mitigate these issues, including Chain-of-Verification (CoVe) [47], concept-level

validation and rectification (EVER) [48], self-refinement [49], and retrieval-augmented knowledge grounding (Re-KGR) [50]. These approaches typically leverage additional LFM invocations for self-reflections [8, 16] or rely on external validation against ground-truth data or knowledge bases [51–55]. However, in mobile task automation, actions of a mobile GUI agent lack a ground truth and its semantic ambiguity makes verification exceptionally difficult. VSA effectively handles this challenge by designing a domain-specific language (DSL, § 4.1) and developer library (§ 4.2) that effectively capture the underlying semantics of these ambiguous GUI actions.

Leveraging Formal Logic for AI Safety. Recent research has explored the use of formal logic to ensure the safety and reliability of AI agents, particularly in areas like planning and control [18–21, 56, 57]. Notably, SELP [20] ensures safe and efficient robot task planning by integrating constrained decoding and domain-specific fine-tuning. Ziyi et al. [19] proposed a safety module for LFM-based robot agents by converting natural language safety constraints into Linear Temporal Logic. However, while previous works typically focus on agents with a pre-defined safety concerns such as map navigation or indoor task planning [18–21], our system targets agents with an enormous number of environment, i.e., mobile apps. To handle different environments of mobile apps, our optimized autoformalization algorithm effectively translates safety requirements for diverse set of mobile tasks.

10 Discussion & Future Work

Piggybacking on App Development Frameworks. The current VSA implementation relies on developer-defined state and transition definition. However, given that modern mobile development frameworks like React Native, Jetpack Compose, and SwiftUI [38–40] already incorporate built-in state management, there is significant potential to "piggyback" on these existing mechanisms. Integrating VSA in this way could streamline the development process and reduce the overhead associated with adopting VSA.

Automated State Definition via LFM. Another approach for simplifying VSA's integration is automating the state definition process using LFMs. Existing GUI agents [4, 6, 7] often employ LFM-powered screen analysis to automatically annotate GUI elements and infer application state. A similar approach could be adapted to identify and define candidate state predicates for VSA. However, while this automation could reduce developer effort, it introduces inherent risks due to the probabilistic nature of LFMs. Relying on potentially inaccurate LFM-derived state definitions could compromise the system's reliability. Therefore, while LFM-based automation offers a potential convenience, the current approach of developer-defined states ensures the precision necessary for a robust verification system.

Automated Predicate Memory Construction. As demonstrated in the evaluation, VSA-Warm's performance significantly benefits from the pre-populated predicate memory. However, the current implementation relies on VSA-Cold to successfully complete tasks before they can be added to the memory, creating a performance bottleneck. One promising approach involves leveraging direct user feedback. By simply asking the user whether a task was completed successfully or not, we can obtain a ground-truth verification result. This user-provided ground truth can then be used to selectively populate the predicate memory. This approach would bypass the VSA-Cold bottleneck and accelerate the creation of a robust and accurate predicate memory, leading to improved overall performance.

Interactive Feedback & Dual Verification While VSA achieves high verification accuracy, it cannot guarantee 100% safety. This is critical in high-risk scenarios like financial transactions where even a single error is unacceptable. To address these limitations, we propose human-in-the-loop strategies. Specifically, we recommend two complementary safeguards: (1) interactive rule-level confirmation that prompts users for confirmation when rules are repeatedly violated, enabling quick human identification of potential False Positives. (2) Dual verification requiring human approval for critical operations (payments, data transfers) regardless of VSA's verification outcome—which guards against potential False Negatives. These mechanisms provide practical safeguards for commercial deployment while preserving VSA's automation benefits.

11 Conclusion

This paper introduced VeriSafe Agent (VSA), a novel logic-based verification system designed to enhance the safety and reliability of Mobile GUI Agents. Through VSA, we have demonstrated that logic-driven verification can effectively safeguard GUI agents against the inherent uncertainties of LFM-based task automation. As AI-driven tasks become increasingly integrated into our daily life, VSA presents a crucial step toward safer use of AIs.

Acknowledgments

This work was partly supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant (RS-2019-II190421, RS-2022-II221045, RS-2025-25442569, RS-2025-02305705, RS-2020-II201819, RS-2023-00232728), the National Research Foundation of Korea(NRF) grant (RS-2024-00338454, RS-2021-NR060080, RS-2025-00522352, RS-2024-00347516), and K-Startup grant (20144069) funded by Korea government (MSIT).

References

- [1] openai. Creating safe agi that benefits all of humanity, 2023. URL <https://openai.com/>.
- [2] anthropic. Talk to claude, 2023. URL <https://claude.ai/>.
- [3] meta. Introducing llama 2, 2023. URL <https://ai.meta.com/llama/>.
- [4] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. Empowering llm to use smartphone for intelligent task automation. *arXiv preprint arXiv:2308.15272*, 2023.
- [5] Hao Wen, Shizuo Tian, Borislav Pavlov, Wenjie Du, Yixuan Li, Ge Chang, Shanhui Zhao, Jiacheng Liu, Yunxin Liu, Ya-Qin Zhang, and Yuanchun Li. Autodroid-v2: Boosting slm-based gui agents via code generation, 2024. URL <https://arxiv.org/abs/2412.18116>.
- [6] Sunjae Lee, Junyoung Choi, Jungjae Lee, Munim Hasan Wasi, Hojun Choi, Steve Ko, Sangeun Oh, and Insik Shin. Mobilegpt: Augmenting llm with human-like app memory for mobile task automation. New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400704895. doi: 10.1145/3636534.3690682. URL <https://doi.org/10.1145/3636534.3690682>.
- [7] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. Appagent: Multimodal agents as smartphone users, 2023. URL <https://arxiv.org/abs/2312.13771>.
- [8] Junyang Wang, Haiyang Xu, Haitao Jia, Xi Zhang, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-agent-v2: Mobile device operation assistant with effective navigation via multi-agent collaboration, 2024. URL <https://arxiv.org/abs/2406.01014>.
- [9] openai. Introducing operator, 2025. URL <https://openai.com/index/introducing-operator/>.
- [10] anthropic. Computer use (beta), 2025. URL <https://docs.anthropic.com/en/docs/agents-and-tools/computer-use>.
- [11] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Gpt-4v(ision) is a generalist web agent, if grounded. In *Proceedings of the 41st International Conference on Machine Learning, ICML'24*. JMLR.org, 2024.
- [12] Christopher Rawles, Sarah Clinckemallie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyi Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents, 2024. URL <https://arxiv.org/abs/2405.14573>.
- [13] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Zhang, Juanzi Li, Bin Xu, Yuxiao Dong, Ming Ding, and Jie Tang. Cogagent: A visual language model for gui agents, 2024. URL <https://arxiv.org/abs/2312.08914>.
- [14] Mingzhe Xing, Rongkai Zhang, Hui Xue, Qi Chen, Fan Yang, and Zhen Xiao. Understanding the weakness of large language model agents within a complex android environment, 2024. URL <https://arxiv.org/abs/2402.06596>.
- [15] Li Zhang, Shihe Wang, Xianqing Jia, Zhihan Zheng, Yunhe Yan, Longxi Gao, Yuanchun Li, and Mengwei Xu. Llamatouch: A faithful and scalable testbed for mobile ui task automation. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, New York, NY, USA, 2024.
- [16] Yanda Li, Chi Zhang, Wanqi Yang, Bin Fu, Pei Cheng, Xin Chen, Ling Chen, and Yunchao Wei. Appagent v2: Advanced agent for flexible mobile interactions, 2024. URL <https://arxiv.org/abs/2408.11824>.
- [17] Yuhuai Wu, Albert Qiaocho Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, *Proceedings of the 36th International Conference on Neural Information Processing Systems*, 2022.
- [18] Jason Xinyu Liu, Ziyi Yang, Ifrah Idrees, Sam Liang, Benjamin Schornstein, Stefanie Tellex, and Ankit Shah. Grounding complex natural language commands for temporal tasks in unseen environments. In *Conference on Robot Learning, CoRL 2023, 6-9 November 2023, Atlanta, GA, USA*, volume 229 of *Proceedings of Machine Learning Research*, pages 1084–1110. PMLR, 2023.
- [19] Ziyi Yang, Shreyas Sundara Raman, Ankit Shah, and Stefanie Tellex. Plug in the safety chip: Enforcing constraints for llm-driven robot agents. In *IEEE International Conference on Robotics and Automation, ICRA 2024, Yokohama, Japan, May 13-17, 2024*, pages 14435–14442. IEEE, 2024.
- [20] Yi Wu, Zikang Xiong, Yiran Hu, Shreyash S. Iyengar, Nan Jiang, Aniket Bera, Lin Tan, and Suresh Jagannathan. SELP: generating safe and efficient task plans for robot agents with large language models. *CoRR*, abs/2409.19471, 2024.
- [21] Ike Obi, Vishnunandan LN Venkatesh, Weizheng Wang, Ruiqi Wang, Dayoon Suh, Temitope I Amosa, Wonse Jo, and Byung-Cheol Min. Safeplan: Leveraging formal logic and chain-of-thought reasoning for enhanced safety in llm-based robotic task planning. *arXiv preprint arXiv:2503.06892*, 2025.
- [22] William Liu, Liang Liu, Yaxuan Guo, Han Xiao, Weifeng Lin, Yuxiang Chai, Shuai Ren, Xiaoyu Liang, Linghao Li, Wenhao Wang, et al. Llm-powered gui agents in phone automation: Surveying progress and prospects. 2025.
- [23] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liquan Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. Large language model-brained gui agents: A survey, 2025. URL <https://arxiv.org/abs/2411.18279>.
- [24] Shuai Wang, Weiwen Liu, Jingxuan Chen, Yuqi Zhou, Weinan Gan, Xingshan Zeng, Yuhang Che, Shuai Yu, Xinlong Hao, Kun Shao, Bin Wang, Chuhan Wu, Yasheng Wang, Ruiming Tang, and Jianye Hao. Gui agents with foundation models: A comprehensive survey, 2025. URL <https://arxiv.org/abs/2411.04890>.
- [25] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, Rui Kong, Yile Wang, Hanfei Geng, Jian Luan, Xuefeng Jin, Zilong Ye, Guanqing Xiong, Fan Zhang, Xiang Li, Mengwei Xu, Zhijun Li, Peng Li, Yang Liu, Ya-Qin Zhang, and Yunxin Liu. Personal llm agents: Insights and survey about the capability, efficiency and security, 2024. URL <https://arxiv.org/abs/2401.05459>.
- [26] Junlin Xie, Zhihong Chen, Ruifei Zhang, Xiang Wan, and Guanbin Li. Large multimodal agents: A survey, 2024. URL <https://arxiv.org/abs/2402.15116>.
- [27] B. Berard, P. McKenzie, M. Bidoit, A. Finkel, F. Laroussinie, A. Petit, L. Petrucci, and P. Schnoebelen. *Systems and Software Verification: Model-Checking Techniques and Tools*. Springer Berlin Heidelberg, 2014. ISBN 9783662045596.
- [28] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. The MIT Press, 2008. ISBN 026202649X.
- [29] Edmund M. Clarke, Orna Grumberg, Daniel Kroening, Doron A. Peled, and Helmut Veith. *Model checking, 2nd Edition*. MIT Press, 2018.
- [30] Bruno Blanchet, Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. A static analyzer for large safety-critical software. *CoRR*, abs/cs/0701193, 2007.
- [31] Thomas Kropf. *Introduction to Formal Hardware Verification: Methods and Tools for Designing Correct Circuits and Systems*. Springer-Verlag, 1st edition, 1999. ISBN 3540654453.
- [32] Nijat Rajabli, Francesco Flammini, Roberto Nardone, and Valeria Vitorini. Software verification and validation of safe autonomous cars: A systematic literature review. *IEEE Access*, 9:4797–4819, 2020.

- [33] Rose Bohrer, Yong Kiam Tan, Stefan Mitsch, Magnus O. Myreen, and André Platzer. Veriphy: verified controller executables from verified cyber-physical system models. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, page 617–630, 2018.
- [34] Stefan Mitsch and André Platzer. Modelplex: Verified runtime validation of verified cyber-physical system models. *Formal Methods in System Design*, 2016.
- [35] Glenford J Myers, Corey Sandler, and Tom Badgett. *The art of software testing*. John Wiley & Sons, 2011.
- [36] Ezio Bartocci, Yliès Falcone, Adrian Francalanza, and Giles Reger. Introduction to runtime verification. *Lectures on Runtime Verification: Introductory and Advanced Topics*, pages 1–33, 2018.
- [37] Leonardo de Moura and Nikolaj Bjørner. Z3: An Efficient SMT Solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2008.
- [38] Meta. State, 2025. URL <https://reactnative.dev/docs/state>.
- [39] Google. State and jetpack compose, 2025. URL <https://developer.android.com/develop/ui/compose/state>.
- [40] Apple. State, 2025. URL <https://developer.apple.com/documentation/swiftui/state>.
- [41] Serge Abiteboul, Richard Hull, and Victor Vianu. *Foundations of Databases: The Logical Level*. Addison-Wesley Longman Publishing Co., Inc., USA, 1st edition, 1995. ISBN 0201537710.
- [42] Leon Sterling and Ehud Shapiro. *The art of Prolog (2nd ed.): advanced programming techniques*. MIT Press, Cambridge, MA, USA, 1994. ISBN 0262193388.
- [43] Temesghen Kahsai, Philipp Rümmer, Huascar Sanchez, and Martin Schäfer. Jayhorn: A framework for verifying java programs. In *Computer Aided Verification*. Springer International Publishing, 2016.
- [44] Arie Gurfinkel, Temesghen Kahsai, Anvesh Komuravelli, and Jorge A. Navas. The seahorn verification framework. In *Computer Aided Verification*, pages 343–361. Springer International Publishing, 2015.
- [45] Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. Rusthorn: Chc-based verification for rust programs. *ACM Trans. Program. Lang. Syst.*, 43(4), 2021.
- [46] Nikolaj Bjørner, Arie Gurfinkel, Ken McMillan, and Andrey Rybalchenko. *Horn Clause Solvers for Program Verification*, pages 24–51. Springer International Publishing, 2015.
- [47] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models, 2023. URL <https://arxiv.org/abs/2309.11495>.
- [48] Haoqiang Kang, Juntong Ni, and Huaxiu Yao. Ever: Mitigating hallucination in large language models through real-time verification and rectification, 2023. URL <https://arxiv.org/abs/2311.09114>.
- [49] Aman Madaan, Niket Tandon, Prakhar Gupta, et al. Self-refine: Iterative refinement with self-feedback, 2023. URL <https://arxiv.org/abs/2303.17651>.
- [50] Mengjia Niu, Hao Li, Jie Shi, Hamed Haddadi, and Fan Mo. Mitigating Hallucinations in Large Language Models via Self-Refinement-Enhanced Knowledge Retrieval, 2024. URL <https://arxiv.org/abs/2405.06545>.
- [51] Sewon Min, Kalpesh Krishna, Ximing Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FactScore: Fine-grained atomic evaluation of factual precision in long form text generation, 2023.
- [52] Ian Chern, Shiyang Chern, Shushan Chen, Weizhi Yuan, Kai Feng, Chunting Zhou, Junxian He, Graham Neubig, Pengfei Liu, et al. FacTool: Factuality detection in generative AI—A tool augmented framework for multi-task and multi-domain scenarios, 2023.
- [53] Feng Nan, Ramesh Nallapati, Zhiling Wang, Cicero Nogueira dos Santos, Huan Zhu, Dong Zhang, Kathy McKeown, and Bing Xiang. Entity-level factual consistency of abstractive text summarization, 2021.
- [54] Ben Goodrich, Vrindavan Rao, Peter J. Liu, and Mina Saleh. Assessing the factual accuracy of generated text. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 166–175, 2019.
- [55] Kurt Shuster, Spencer Poff, M. Chen, Douwe Kiela, and Jason Weston. Retrieval augmentation reduces hallucination in conversation, 2021.
- [56] Francesco Fuggitti and Tathagata Chakraborti. NL2LTL - a python package for converting natural language (NL) instructions to linear temporal logic (LTL) formulas. In Brian Williams, Yiling Chen, and Jennifer Neville, editors, *Thirty-Seventh AAAI Conference on Artificial Intelligence*, AAAI, pages 16428–16430. AAAI Press, 2023.
- [57] Matthias Cosler, Christopher Hahn, Daniel Mendoza, Frederik Schmitt, and Caroline Trippel. nl2spec: Interactively translating unstructured natural language to temporal logics with large language models. In Constantin Enea and Akash Lal, editors, *Computer Aided Verification - 35th International Conference, CAV 2023, Paris, France, July 17-22, 2023, Proceedings, Part II*, volume 13965 of *Lecture Notes in Computer Science*, pages 383–396. Springer, 2023.